TRAIN COACH MANAGEMENT SYSTEM

Doubly Linked List and Circular Linked List

1. Doubly Linked List (DLL)

I. Problem Statement

Design and implement a menu-driven Train Coach Management System in C++ using a Doubly Linked List. Each coach stores a unique coach number and coach type. The system must support insertion, deletion, searching, forward and backward display, and counting of coaches.

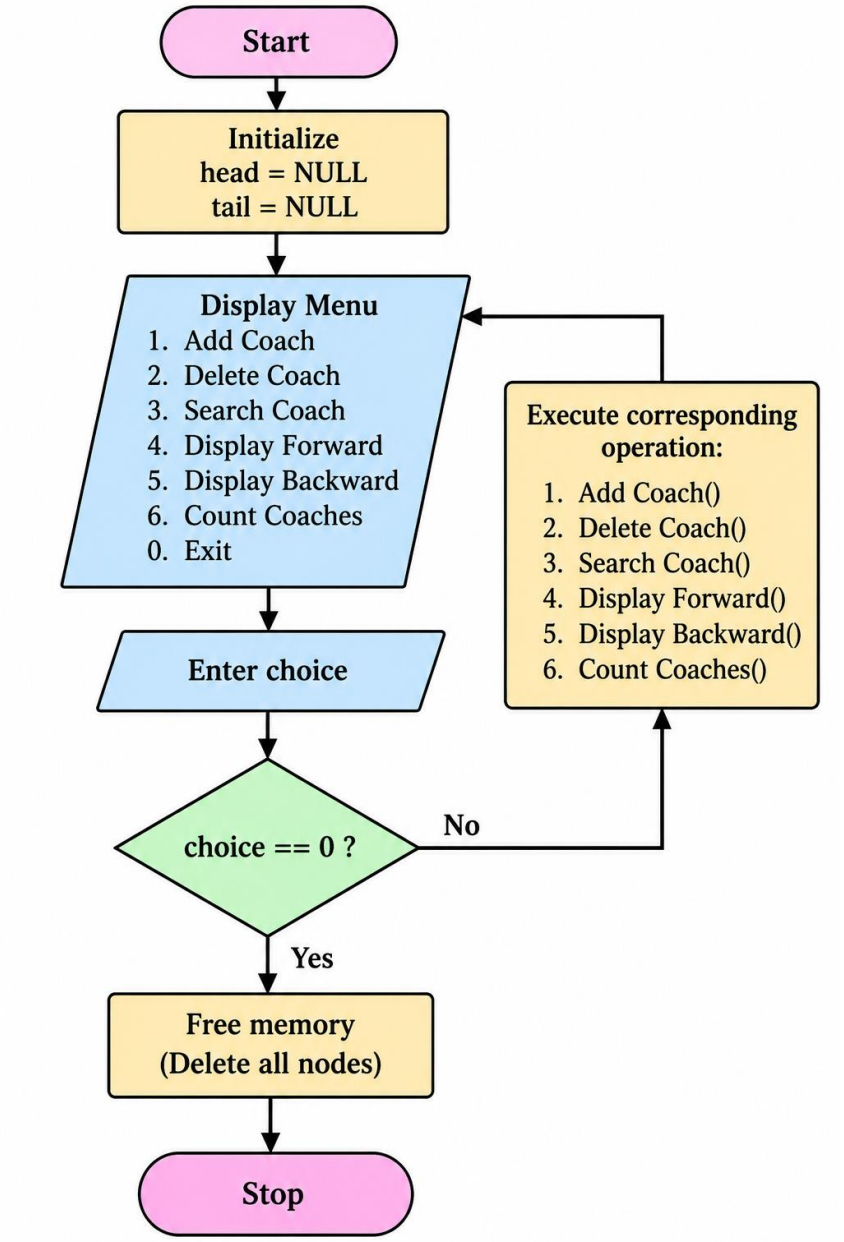
II. Objectives

- To understand the structure and operation of a Doubly Linked List.
- To create coach nodes dynamically using pointers.
- To insert and delete coach records while maintaining both prev and next links.
- To search for a coach using its coach number.
- To traverse and display the train in forward and reverse directions.
- To count the coaches and release dynamically allocated memory safely.

III. Algorithm

1. Start the program and initialize head and tail to NULL.
2. Display the menu containing Add, Delete, Search, Display Forward, Display Backward, Count, and Exit.
3. Read the user's choice.
4. If the choice is Add, create a new coach node and read its coach number.
5. Check the list for a duplicate coach number. If it exists, delete the unused node and return to the menu.
6. Read the coach type. Set the new node's prev and next pointers to NULL.
7. If the list is empty, set both head and tail to the new node. Otherwise, link the current tail to the new node and update tail.
8. If the choice is Delete, search for the requested coach number. If found, reconnect its previous and next nodes, update head or tail when required, and delete the node.
9. If the choice is Search, traverse from head and compare each coach number with the requested number. Display the result.
10. If Display Forward is selected, traverse from head to NULL and print every coach.
11. If Display Backward is selected, traverse from tail to NULL using prev pointers and print every coach.
12. If Count is selected, traverse from head to NULL and increment a counter for each node.
13. Repeat the menu until the user selects Exit.
14. On Exit, delete all remaining nodes to free memory, then stop.

IV. Flowchart



V. Code

```
#include <iostream>
#include <string>
using namespace std;

struct Coach {
    int coachNo;
    string coachType;
    Coach* prev;
    Coach* next;
};

Coach* head = NULL;
Coach* tail = NULL;

void addCoach() {
    Coach* newCoach = new Coach;

    cout << "Enter coach number: ";
    cin >> newCoach->coachNo;

    Coach* temp = head;
    while (temp != NULL) {
        if (temp->coachNo == newCoach->coachNo) {
            cout << "Coach number already exists!\n";
            delete newCoach;
            return;
        }
        temp = temp->next;
    }

    cout << "Enter coach type (AC/SL/GEN): ";
    cin >> newCoach->coachType;

    newCoach->prev = NULL;
    newCoach->next = NULL;

    if (head == NULL) {
        head = tail = newCoach;
    } else {
        tail->next = newCoach;
        newCoach->prev = tail;
        tail = newCoach;
    }

    cout << "Coach added successfully!\n";
}

void deleteCoach() {
    if (head == NULL) {
        cout << "No coaches available!\n";
        return;
    }

    int number;
    cout << "Enter coach number to delete: ";
    cin >> number;

    Coach* temp = head;
    while (temp != NULL && temp->coachNo != number)
        temp = temp->next;

    if (temp == NULL) {
        cout << "Coach not found!\n";
        return;
    }

    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else
```

```
        head = temp->next;

    if (temp->next != NULL)
        temp->next->prev = temp->prev;
    else
        tail = temp->prev;

    delete temp;
    cout << "Coach deleted successfully!\n";
}

void searchCoach() {
    int number;
    cout << "Enter coach number to search: ";
    cin >> number;

    Coach* temp = head;
    while (temp != NULL) {
        if (temp->coachNo == number) {
            cout << "Coach found: " << temp->coachNo
                 << " - " << temp->coachType << '\n';
            return;
        }
        temp = temp->next;
    }

    cout << "Coach not found!\n";
}

void displayForward() {
    if (head == NULL) {
        cout << "No coaches available!\n";
        return;
    }

    Coach* temp = head;
    cout << "\nForward Train: ";

    while (temp != NULL) {
        cout << "[" << temp->coachNo << " "
             << temp->coachType << "] <-> ";
        temp = temp->next;
    }

    cout << "NULL\n";
}

void displayBackward() {
    if (tail == NULL) {
        cout << "No coaches available!\n";
        return;
    }

    Coach* temp = tail;
    cout << "\nBackward Train: ";

    while (temp != NULL) {
        cout << "[" << temp->coachNo << " "
             << temp->coachType << "] <-> ";
        temp = temp->prev;
    }

    cout << "NULL\n";
}

void countCoaches() {
    int count = 0;
    Coach* temp = head;

    while (temp != NULL) {
        count++;
```

```
        temp = temp->next;
    }

    cout << "Total coaches: " << count << '\n';
}

void freeMemory() {
    while (head != NULL) {
        Coach* temp = head;
        head = head->next;
        delete temp;
    }

    tail = NULL;
}

int main() {
    int choice;

    do {
        cout << "\n--- TRAIN COACH MANAGEMENT (DLL) ---\n";
        cout << "1. Add Coach\n";
        cout << "2. Delete Coach\n";
        cout << "3. Search Coach\n";
        cout << "4. Display Forward\n";
        cout << "5. Display Backward\n";
        cout << "6. Count Coaches\n";
        cout << "0. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                addCoach();
                break;

            case 2:
                deleteCoach();
                break;

            case 3:
                searchCoach();
                break;

            case 4:
                displayForward();
                break;

            case 5:
                displayBackward();
                break;

            case 6:
                countCoaches();
                break;

            case 0:
                freeMemory();
                cout << "Exiting program...\n";
                break;

            default:
                cout << "Invalid choice!\n";
        }
    } while (choice != 0);

    return 0;
}
```

VI. Code Output

```
--- TRAIN COACH MANAGEMENT (DLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Forward
5. Display Backward
6. Count Coaches
0. Exit
Enter choice: 1
Enter coach number: 1
Enter coach type (AC/SL/GEN): AC
Coach added successfully!

--- TRAIN COACH MANAGEMENT (DLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Forward
5. Display Backward
6. Count Coaches
0. Exit
Enter choice: 1
Enter coach number: 2
Enter coach type (AC/SL/GEN): SL
Coach added successfully!

--- TRAIN COACH MANAGEMENT (DLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Forward
5. Display Backward
6. Count Coaches
0. Exit
Enter choice: 1
Enter coach number: 3
Enter coach type (AC/SL/GEN): GEN
Coach added successfully!

--- TRAIN COACH MANAGEMENT (DLL) ---
```

```
--- TRAIN COACH MANAGEMENT (DLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Forward
5. Display Backward
6. Count Coaches
0. Exit
Enter choice: 2
Enter coach number to delete: 1
Coach deleted successfully!

--- TRAIN COACH MANAGEMENT (DLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Forward
5. Display Backward
6. Count Coaches
0. Exit
Enter choice: 4

Forward Train: [2 SL] <-> [3 GEN] <-> NULL

--- TRAIN COACH MANAGEMENT (DLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Forward
5. Display Backward
6. Count Coaches
0. Exit
Enter choice: 5

Backward Train: [3 GEN] <-> [2 SL] <-> NULL

--- TRAIN COACH MANAGEMENT (DLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Forward
```

```
--- TRAIN COACH MANAGEMENT (DLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Forward
5. Display Backward
6. Count Coaches
0. Exit
Enter choice: 6
Total coaches: 2

--- TRAIN COACH MANAGEMENT (DLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Forward
5. Display Backward
6. Count Coaches
0. Exit
Enter choice: 0
Exiting program...

--------------------------------
Process exited after 118.1 seconds with return value 0
Press any key to continue . . .
```

2. Circular Linked List (CLL)

I. Problem Statement

Design and implement a menu-driven Train Coach Management System in C++ using a Circular Singly Linked List. Each coach stores a unique coach number and coach type. The last coach must point back to the first coach. The system must support insertion, deletion, searching, display, and counting of coaches.

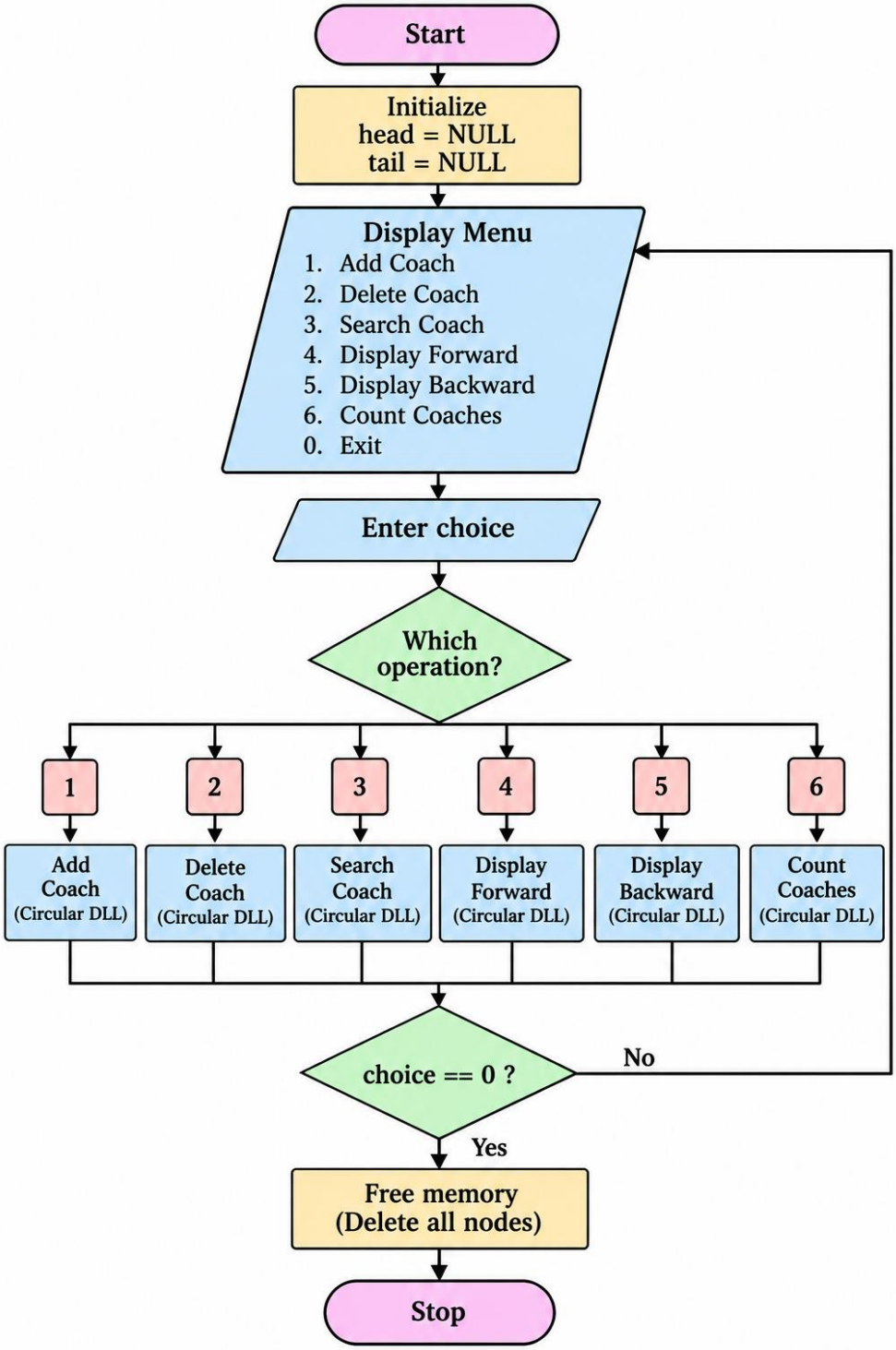
II. Objectives

- To understand the structure and operation of a Circular Linked List.
- To create coach nodes dynamically using pointers.
- To insert and delete coach records while preserving the circular link.
- To search for a coach using its coach number.
- To display all coaches by stopping when traversal returns to the first node.
- To count the coaches and release allocated memory safely.

III. Algorithm

1. Start the program and initialize tail to NULL. The first coach, when present, is tail->next.
2. Display the menu containing Add, Delete, Search, Display, Count, and Exit.
3. Read the user's choice.
4. If the choice is Add, create a new node and read its coach number.
5. If the list is not empty, traverse the circle from tail->next and reject a duplicate coach number if found.
6. Read the coach type. If the list is empty, set tail to the new node and make its next pointer point to itself.
7. Otherwise, link the new node between tail and the first node, then update tail to the new node.
8. If the choice is Delete, begin at the first node and track the previous node while searching for the requested coach number.
9. If the matching node is the only node, set tail to NULL. Otherwise, bypass the node; if it is tail, update tail to the previous node. Delete the removed node.
10. If the choice is Search, traverse the circle and compare coach numbers until the coach is
11. found or the pointer returns to the first node.
12. If Display is selected, print each coach using a do-while loop until traversal returns to the first node.
13. If Count is selected, traverse the circle once and increment the counter for every node.
14. Repeat the menu until the user selects Exit.
15. On Exit, temporarily break the circular link, delete all nodes, and stop.

IV. Flowchart



V. C++ Code

```
#include <iostream>
#include <string>
using namespace std;
struct Coach {
    int coachNo;
    string coachType;
    Coach* next;
};

Coach* head = NULL;

void addCoach() {
    Coach* newCoach = new Coach;

    cout << "Enter coach number: ";
    cin >> newCoach->coachNo;

    Coach* temp = head;

    if (head != NULL) {
        do {
            if (temp->coachNo == newCoach->coachNo) {
                cout << "Coach number already exists!\n";
                delete newCoach;
                return;
            }
            temp = temp->next;
        } while (temp != head);
    }

    cout << "Enter coach type (AC/SL/GEN): ";
    cin >> newCoach->coachType;

    if (head == NULL) {
        head = newCoach;
        newCoach->next = head;
    } else {
        temp = head;

        while (temp->next != head) {
            temp = temp->next;
        }

        temp->next = newCoach;
        newCoach->next = head;
    }

    cout << "Coach added successfully!\n";
}

void deleteCoach() {
    if (head == NULL) {
        cout << "No coaches available!\n";
        return;
    }

    int number;
    cout << "Enter coach number to delete: ";
    cin >> number;

    Coach* temp = head;
    Coach* prev = NULL;

    do {
        if (temp->coachNo == number)
            break;

        prev = temp;
        temp = temp->next;
```

```
    } while (temp != head);

    if (temp->coachNo != number) {
        cout << "Coach not found!\n";
        return;
    }

    if (temp->next == head && temp == head) {
        head = NULL;
    } else {
        if (temp == head) {
            Coach* last = head;

            while (last->next != head) {
                last = last->next;
            }

            head = head->next;
            last->next = head;
        } else {
            prev->next = temp->next;
        }
    }

    delete temp;
    cout << "Coach deleted successfully!\n";
}

void searchCoach() {
    if (head == NULL) {
        cout << "No coaches available!\n";
        return;
    }

    int number;
    cout << "Enter coach number to search: ";
    cin >> number;

    Coach* temp = head;

    do {
        if (temp->coachNo == number) {
            cout << "Coach found: " << temp->coachNo
                 << " - " << temp->coachType << '\n';
            return;
        }

        temp = temp->next;
    } while (temp != head);

    cout << "Coach not found!\n";
}

void displayCoaches() {
    if (head == NULL) {
        cout << "No coaches available!\n";
        return;
    }

    Coach* temp = head;

    cout << "\nCircular Train: ";

    do {
        cout << "[" << temp->coachNo << " "
             << temp->coachType << "] -> ";
        temp = temp->next;
    } while (temp != head);

    cout << "(Back to Head)\n";
}
```

```
void countCoaches() {
    int count = 0;

    if (head != NULL) {
        Coach* temp = head;

        do {
            count++;
            temp = temp->next;
        } while (temp != head);
    }

    cout << "Total coaches: " << count << '\n';
}

void freeMemory() {
    if (head == NULL)
        return;

    Coach* temp = head->next;

    while (temp != head) {
        Coach* nextCoach = temp->next;
        delete temp;
        temp = nextCoach;
    }

    delete head;
    head = NULL;
}

int main() {
    int choice;

    do {
        cout << "\n--- TRAIN COACH MANAGEMENT (CLL) ---\n";
        cout << "1. Add Coach\n";
        cout << "2. Delete Coach\n";
        cout << "3. Search Coach\n";
        cout << "4. Display Coaches\n";
        cout << "5. Count Coaches\n";
        cout << "0. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                addCoach();
                break;

            case 2:
                deleteCoach();
                break;

            case 3:
                searchCoach();
                break;

            case 4:
                displayCoaches();
                break;

            case 5:
                countCoaches();
                break;

            case 0:
                freeMemory();
                cout << "Exiting program...\n";
                break;
```

```
            default:
                cout << "Invalid choice!\n";
        }
    } while (choice != 0);

    return 0;
}
```

VI. Code Output

```
--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Coaches
5. Count Coaches
0. Exit
Enter choice: 1
Enter coach number: 1
Enter coach type (AC/SL/GEN): AC
Coach added successfully!

--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Coaches
5. Count Coaches
0. Exit
Enter choice: 1
Enter coach number: 2
Enter coach type (AC/SL/GEN): SL
Coach added successfully!

--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Coaches
5. Count Coaches
0. Exit
Enter choice: 1
Enter coach number: 3
Enter coach type (AC/SL/GEN): GEN
Coach added successfully!

--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Coaches
5. Count Coaches
0. Exit
Enter choice: 2
Enter coach number to delete: 3
Coach deleted successfully!

--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Coaches
5. Count Coaches
0. Exit
Enter choice: 3
Enter coach number to search: 1
Coach found: 1 - AC

--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Coaches
5. Count Coaches
0. Exit
Enter choice: 4

Circular Train: [1 AC] -> [2 SL] -> (Back to Head)

--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Coaches
5. Count Coaches
0. Exit
--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Coaches
5. Count Coaches
0. Exit
Enter choice: 5
Total coaches: 2

--- TRAIN COACH MANAGEMENT (CLL) ---
1. Add Coach
2. Delete Coach
3. Search Coach
4. Display Coaches
5. Count Coaches
0. Exit
Enter choice: 0
Exiting program...

--------------------------------
Process exited after 65.89 seconds with return value 0
Press any key to continue . . .
```